

A3-PS027-TC190

IPv6 Website and Dependency Readiness Auditor

AIORI Hackathon 3

October 11, 2026

Problem Description

An AAAA record on the apex domain does not make a website IPv6-ready.

- **IPv6-only impact.** On IPv6-only networks, IPv4-only scripts, styles, fonts, APIs and tags fail to load, leaving broken layouts and hung pages.
- **Happy Eyeballs masking.** On dual-stack links, RFC 8305 [2] falls back to IPv4 after a short delay (250 ms by default). Operators never see the broken IPv6 path, yet users pay the latency.
- **Our approach.** Load each page in a headless browser, map its full runtime dependency tree, and test every host through four isolated stages: DNS, TCP, TLS and HTTP.
- **Validation.** A controlled testbed seeded with known faults (target: 100% detection), then an audit of 30 real websites across six sectors.

Background

Indian users are largely IPv6-ready; the web they reach is not.

- A modern portal is a multi-origin application: one government or e-commerce page may load assets from 15 or more third-party domains (CDNs, ad networks, payment gateways, authentication providers).
- A DNS answer says nothing about firewalls, routing blackholes, TLS name mismatches or HTTP 5xx errors on the IPv6 path.
- Dynamic scripts, font-face rules, XHR and fetch calls and iframe contents appear only when JavaScript runs in a real browser, so we crawl with a headless browser [5].
- RFC 8200 [1] defines IPv6; RFC 8305 [2] and RFC 6555 [3] define the fallback behaviour we must bypass to measure honestly.

Literature & RFC Review

Standards define IPv6 and the fallback we must work around; the closest tool leaves stage isolation open.

Standard or tool	Focus area	Key insight and limitation
RFC 8200 [1]	IPv6 base specification	Defines packet headers and the addressing architecture. A foundation, not a measurement method.
RFC 8305 [2]	Happy Eyeballs v2	Prefers IPv6, falls back to IPv4 after a short delay. Hides IPv6 failures from browser-based tests.
SAP webres6 [4]	Selenium-based IPv6 checker	A pioneering tool. In our review it is exposed to Happy Eyeballs bias in Chromium, scans serially and does not isolate the failing stage.
W3C Resource Timing [5]	Browser performance API	Standard entries, initiator types and waterfall timestamps. Used to cross-check what the crawler observes.

Proposed Solution

Browser instrumentation plus raw-socket probes judge each dependency by what the IPv6 path really does.

- **Headless Chromium (CDP)** renders the page and records every request with its initiator: parser, script or fetch.
- **Decoupled probing.** Raw IPv4 and IPv6 (AF_INET6) sockets bypass the browser's fallback.
- **Four sequential stages** pin a failure to one layer: **1 DNS** (A/AAAA, NODATA, CNAME, NXDOMAIN, SERVFAIL), **2 TCP** (timeout, refused, no route), **3 TLS** (SNI; expired, untrusted, mismatched), **4 HTTP** (GET/HEAD, TTFB, 4xx/5xx).
- **Six result classes** against the IPv4 baseline: FULL_IPV6_READY, IPV4_ONLY_DEPENDENCY, BROKEN_IPV6_ROUTING, BROKEN_IPV6_TLS, BROKEN_IPV6_HTTP, GLOBAL_FAILURE.

Optimization Proposed

Five changes separate this auditor from an apex-domain check or a browser-only scan.

Dimension	Before (traditional)	After (proposed)
Inspection scope	Apex-domain AAAA check only	Full browser DOM execution: runtime JS, CSS, fonts and third-party APIs
Measurement reliability	Chromium Happy Eyeballs hides failures behind IPv4 fallback	Pure IPv6 and IPv4 socket probes run side by side
Failure classification	Pass or fail, or a generic connection error	Names the failing layer: DNS, TCP, TLS or HTTP
Dependency visualization	Flat list of hostnames	Interactive causal graph with the blast radius of each failure
Performance	Serial crawl, one resource at a time	Concurrent asyncio probing; timings measured in the study

Solution Architecture & Design

One audit flows left to right through the auditor service; the protocol engine is the core.

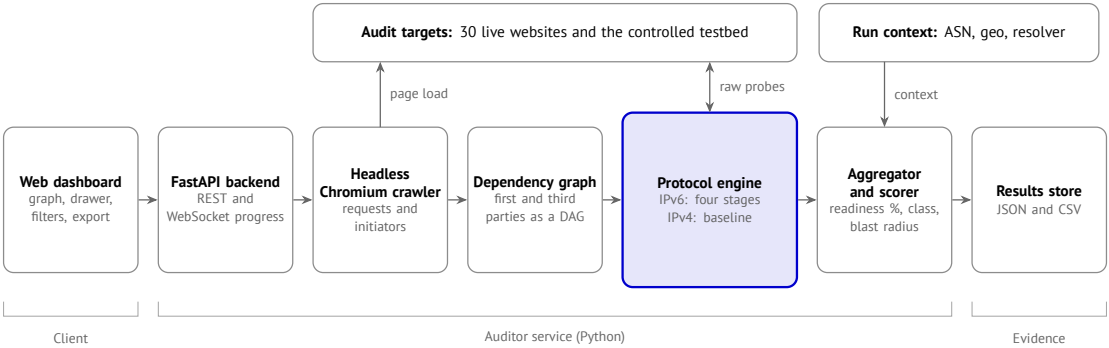


Figure: System Architecture Diagram (component view)

Data Flow Diagram

Five processes turn one URL into a scored, exportable report.

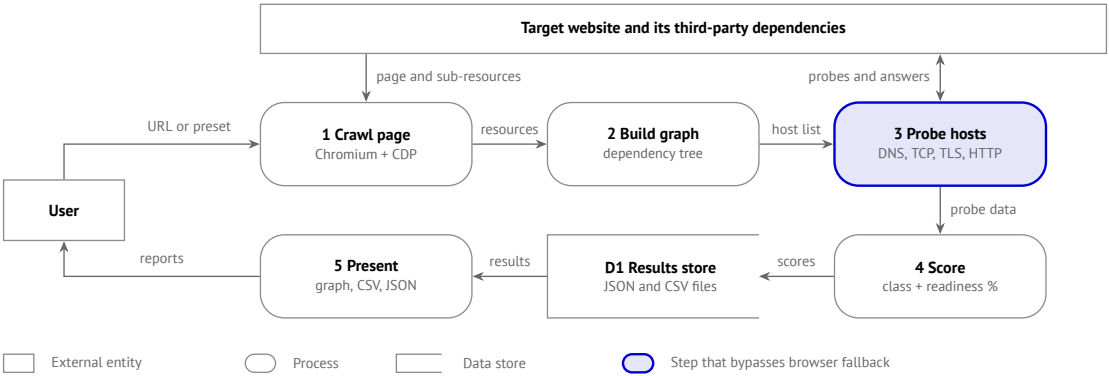


Figure: Data Flow Diagram (DFD)

Deployment Timeline

Level 1 closes on Oct 14; the finals at IIT Jammu are on Dec 17.

Date	Milestone
Oct 14	Level 1 steps 1–7 due
Oct 16	Level 2 teams announced
Oct 16–31	Mentoring sessions, steps 8–10
Oct 31–Nov 4	Evaluators connect with teams
Nov 5	Regional teams announced
Nov 24–30	Regional hackathon (Bengaluru, Pune, Kolkata, Noida)
Dec 17	Finals, IIT Jammu (8 teams)

Methodology

- **Level 1 (steps 1–7):** repository named A3-PS027-TC190, AIORI added as collaborator and given ownership, PDF deck, YouTube introduction video and its link.
- **Level 2 (steps 8–10):** mentoring sessions, code submission, live demo link.
- Build the interceptor, four-stage engine and testbed during mentoring, and have the 30-site results ready by Oct 31.

Bibliography

[1] S. Deering and R. Hinden. “Internet Protocol, Version 6 (IPv6) Specification”. RFC 8200, STD 86, IETF (July 2017).

[2] D. Schinazi and T. Pauly. “Happy Eyeballs Version 2: Better Connectivity Using Concurrency”. RFC 8305, IETF (December 2017).

[3] D. Wing and A. Yourtchenko. “Happy Eyeballs: Success with Dual-Stack Hosts”. RFC 6555, IETF (April 2012).

[4] SAP SE. “webres6”. GitHub repository, github.com/SAP/webres6.

[5] W3C Web Performance Working Group. “Resource Timing”. W3C specification.

[6] AIORI. “AIORI-3 Online Hackathon Guidelines”. portal.aiori.in/aiori-3/hackathon-guideline/ (October 2026).

Thank You for Your Attention!